Exploring Intelligent Dynamic Resource Provisioning for Elastic Massive MIMO vRAN

Parthiban Annamalai[+], Minsung Kim[⋆], Anurag Khandelwal[+], Lin Zhong[+]
[+]Yale University, [⋆]Rutgers University

ABSTRACT

The computational load in a Virtualized Radio Access Network (vRAN) varies rapidly and massive Multiple-Input Multiple-Output (MIMO) makes it even more dynamic. Existing massive MIMO vRAN systems commonly adopt static resource provisioning and thus often result in either under- or over-provisioning of compute resources. In this paper, we explore the feasibility of a *Dynamic Resource Provisioning* (DRP) in a massive MIMO vRAN to dynamically allocate Central Processing Unit (CPU) cores for varying computational loads. We investigate a Reinforcement Learning (RL)-based DRP algorithm that provisions optimal number CPU cores in an automated manner. To experimentally demonstrate its viability, we integrate our RL-based DRP into a real-time fully-software massive MIMO vRAN prototype system. Our early experimental results, however, do not show any significant advantage of an RL-based solution over a simple Steepest Gradient Descent (SGD)-based DRP. We believe that this surprising result is largely due to the fact that current open-source vRAN systems cannot sufficiently reflect the highly complex nature of the real vRAN. We further discuss the way ahead, challenges, and opportunities in this research direction.

CCS CONCEPTS

• **Networks → Mobile networks**.

KEYWORDS

Virtual Radio Access Networks; Massive MIMO; Dynamic Resource Provisioning; Reinforcement Learning

ACM Reference Format:
Parthiban Annamalai, Minsung Kim, Anurag Khandelwal, Lin Zhong. 2025. Exploring Intelligent Dynamic Resource Provisioning for Elastic Massive MIMO vRAN . In *The 26th International Workshop on Mobile Computing Systems and Applications (HOTMOBILE '25), February 26–27, 2025, La Quinta, CA, USA.* ACM, New York, NY, USA, 6 pages. https://doi.org/10.1145/3708468.3711876

1 INTRODUCTION

The Radio Access Network (RAN) is a major component of the cellular system that wirelessly connects client devices to the network. Virtualized RAN (vRAN) is a promising RAN architecture that disaggregates hardware and software by virtualizing network functions to improve operational efficiency and flexibility for Mobile Network Operators (MNOs). vRAN can further help forward cellular baseband processing from the proprietary processing infrastructure at cell sites to a shared (cloud) platform. This architectural evolution not only makes vRAN more scalable and flexible, but also provides an opportunity to deliver faster innovations.

For next-generation wireless networks, massive Multiple-Input Multiple-Output (MIMO) is a key enabling physical layer technology to improve the overall data rate. With massive MIMO, a single base station can simultaneously serve multiple client devices on the same frequency resource using parallel spatial layers (*i.e.,* improved spectral efficiency). The overall network throughput can increase as more spatial layers are enabled to serve many client devices at the same time. However, the computational load in vRAN is proportional to the number of spatial layers enabled. In real-world massive MIMO vRAN, the number of spatial layers is decided based on multiple factors such as user traffic demands, user density, Quality of Service (QoS) parameters, Signal-to-Noise Ratio (SNR) and wireless channel conditions, and these factors rapidly vary. Thus, dynamically varying computational load is considered one of the key challenges for resource provisioning in a massive MIMO vRAN.

Existing vRAN systems commonly deploy manual and fixed resource provisioning schemes [3, 5, 9]. These static strategies often result in either under-provisioning or over-provisioning of compute resources. In the case of under-provisioning (*i.e.,* allocating fewer compute resources than required), the essential baseband processing might not be completed within the allowed processing time deadline. On the other hand, over-provisioning (*i.e.,* allocating more compute resources than required) results in the wastage of compute resources. Given the dynamic computational load on massive MIMO vRANs, it will soon become essential to adopt a *Dynamic Resource Provisioning* (DRP) design that adaptively provisions the computational resources for resource- and cost-efficient vRAN systems.

The broad goal of this paper is to explore the feasibility of intelligent DRP algorithms for massive MIMO vRAN. Particularly, our objective is to provide a preliminary assessment of an *Reinforcement Learning* (RL)-based design that operates in a fully automated manner. Although there were previous attempts to model the computational behavior of vRAN for resource provisioning [1, 4, 10], they showed a rather limited ability to accurately capture this dynamic computational load, even without MIMO support, due to the highly complex nature of the cellular system with many interdependent parameters. Unlike such model-based learning methods, RL is a *model-free* approach, where the RL agent learns while continuously interacting with the underlying system, rather than explicitly modeling its behavior. For this reason, RL is commonly applied for complex systems that include many dependencies [7]. This unique feature motivates us to investigate RL-based DRP algorithms for massive MIMO vRAN.

To experimentally test the viability, we design an RL-based DRP

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
HOTMOBILE '25, February 26–27, 2025, La Quinta, CA, USA
© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-1403-0/25/02…$15.00
https://doi.org/10.1145/3708468.3711876

module written in Rust (§3) and integrate it into Agora [3], a real-time fully software-based realization of massive MIMO vRAN, where the DRP scheme dynamically provisions Central Processing Unit (CPU) cores for optimal processing latency. Unlike a real vRAN that adapts many of its parameters in scheduling granularity to meet varying traffic demands and wireless channel dynamics, there is no such support in Agora. Hence, as a first step, we introduce a new feature in Agora to vary the number of spatial layers across scheduling intervals and thus its computational load dynamically. The proposed RL-based DRP algorithm tries to learn this dynamic behavior and provisions CPU cores accordingly. As a comparison scheme, we design a simple *Steepest Gradient Descent* (SGD)-based DRP algorithm and compare it against the RL-based design.

Our preliminary experimental results are encouraging. DRP for massive MIMO vRAN has promising compute resource gains, compared to conventional static and manual schemes, even on Agora, whose real-time dynamic characteristic is still far from that of a real-world vRAN. **However, the RL-based solution does not show any significant advantage compared to the simpler SGD design.** We observe that while both RL and SGD algorithms can successfully identify the optimal number of CPU cores, simple SGD design features a much shorter allocation time during deployment and less memory requirement. We believe that this happens mainly because the current DRP for Agora is *not complex* enough, and thus even a simple SGD can solve the DRP problem well using the *convex* relationship between the cores and the latency (§4). Nevertheless, we expect that with a degree of dynamic characteristic to real-world vRAN, simple SGD would start performing poorly at some point, and therefore more resilient and comprehensive designs such as RL are required. However, more experimental studies and validations are necessary to quantify potential gains. We discuss this further in this paper, introducing the challenges and opportunities in this research direction, including how to improve RL-based solutions.

2 BACKGROUND AND MOTIVATION

Agora [3] is an open-source fully software-based massive MIMO vRAN prototype system on a single, multi-core server. This framework demonstrated how general-purpose processors can be efficiently utilized to support the demanding computational needs of massive MIMO functionalities in real time. The key design principle is to use all available CPU cores to process the earliest *Transmission Time Interval* (TTI) and minimize its processing time.

Limitations of Static Resource Provisioning. Current vRAN systems, including Agora, adopt a static and manual resource provisioning scheme that allocates a fixed number of CPU cores. These CPU cores are decided manually at the beginning based on the cellular settings, and they remain the same throughout the execution. As discussed earlier, this static allocation often results in either under- or over-provisioning of compute resources and cannot adapt to varying computational loads.

To experimentally demonstrate its limitations, we plot the Cumulative Distribution Function (CDF) of the TTI processing latency in Figure 1 for different numbers of CPU cores allocated for a specific cellular configuration in Agora. Since cellular systems schedule users with TTI granularity, complete baseband processing must be completed within the TTI duration for real-time processing. Thus, we choose the optimal latency range such that it reserves 20% of

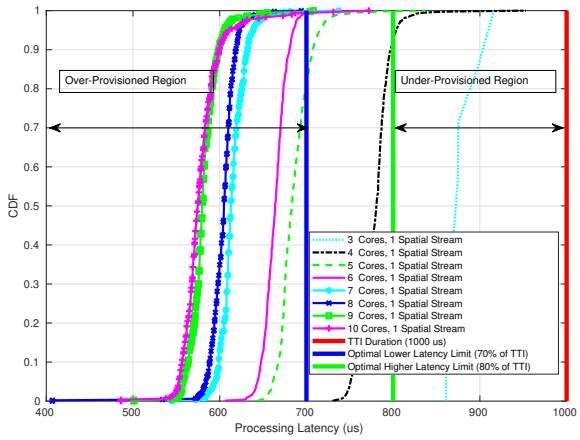

Figure 1: CDF of TTI processing latency for the varying number of CPU cores. The figure shows that proper resource provisioning is essential for efficient vRAN to avoid under- or over-provisioning. Massive MIMO with dynamic spatial layers motivates this further.

available compute power in every TTI for processing spillover. It can be observed that, for this specific fixed configuration, four CPU cores are found to be optimal, while allocating more cores results in wastage of compute resources, and fewer cores might result in losing real-time performance. Moreover, when cellular configurations, such as spatial layers, change across TTIs to support dynamic throughput demands in a massive MIMO vRAN, static resource provisioning cannot avoid severe inefficiencies in the system.

3 AN EXPLORATORY STUDY

In this paper, we explore an RL-based framework for DRP that adaptively allocates optimal CPU cores in an automated manner to a massive MIMO vRAN system based on load variations. Unlike model-based approaches [1, 10], this RL framework characterizes the behavior of the vRAN system using interactive learning in the training phase. We implement an RL DRP scheme on Agora (§2) for its experimental validation. This section is structured as follows.

Section 3.1 describes the mapping of the DRP problem as an RL problem, followed by the learning algorithm design in Section 3.2. Then, Section 3.3 explains the integration of DRP into Agora with implementation details, and Section 3.4 presents a comparison scheme, a simpler SGD-based DRP design.

3.1 Mapping DRP as an RL Problem

Let the underlying system, called environment in the RL context, be in state s at any given time instance. The RL agent continuously interacts with the environment and takes different actions. In general, the action a taken by the RL agent transitions the state from s to s' and receives a reward r with the goal of maximizing the cumulative reward [11]. The DRP is mapped as an RL problem in Figure 2 and the key concepts used are introduced below.

State Space. Our initial state space S contains $C \cdot U \cdot L$ number of states in total, where C is the maximum number of CPU cores available for provisioning, U is the maximum number of spatial layers, and L is the maximum number of latency levels mapped

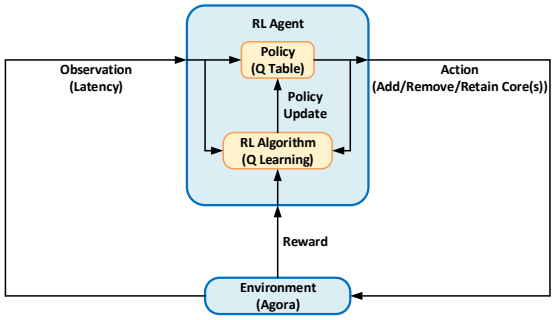

Figure 2: Schematic of mapping DRP as an RL Problem. The RL agent takes action to update CPU cores, which impacts the TTI processing latency. A mapped latency value and reward received help the RL agent to update its Q Table.

from absolute processing latency. A specific state s is coded as,

$$s = c + u \cdot C + l \cdot U \cdot C, \tag{1}$$

where c represents the number of CPU cores assigned (mapped from 0 to C-1), u does the number of spatial layers (from 0 to U-1), and l does the latency level (from 0 to L-1) in the current TTI. This mapping encodes possible feature combinations into unique states.

Action Space. An RL agent can take three different actions, namely add a CPU core, remove a CPU core, or retain the same number of CPU cores, *i.e.,* without a change in CPU cores.

Rewards. In any state s, as derived in Equation 1, an RL agent learns the environment (Agora in our case) by taking three different actions described above. These actions have an impact on the TTI processing latency, which is the time required to complete the baseband processing in a TTI. Our objective is to find the optimal number of CPU cores such that the resulting TTI processing latency lies within the optimal range marked in Figure 1. When the resultant latency lies outside this range, a proportionally higher negative reward is given to the RL agent penalizing its wrong action.

3.2 Q-Learning Algorithm Design

Q-Learning is a model-free RL algorithm that learns the Q-value of an action in a particular state to maximize the reward. We use Bellman's Equation [11] to determine this Q-value based on the state s, action a and reward r received for the action and to deduce how good it is to be in that state. Q-Table is formed as a 2-D matrix and the learning algorithm updates its values for all possible states defined in Equation 1.

Training. For every Episode, the RL agent initially allocates arbitrarily chosen CPU cores to Agora. Using this CPU core information, its corresponding mapped latency, and the number of spatial layer details retrieved from Agora, the state s is determined using Equation 1. For state s, max. Q-value from Q-Table determines the best action a. This action may transition Agora to a new state, and the respective Q-value is updated using Bellman's Equation, which completes an Epoch within an Episode. This process is repeated for many Epochs until reaching the optimal latency range or terminal count initialized. This process continues for all configured Episodes to possibly cover all states. The decreasing Epoch count across Episodes means that the Q-Table is getting trained sufficiently. At the end of sufficient training, the expectation is that the necessary information required to choose an optimal action for every state is self-contained in Q-Table.

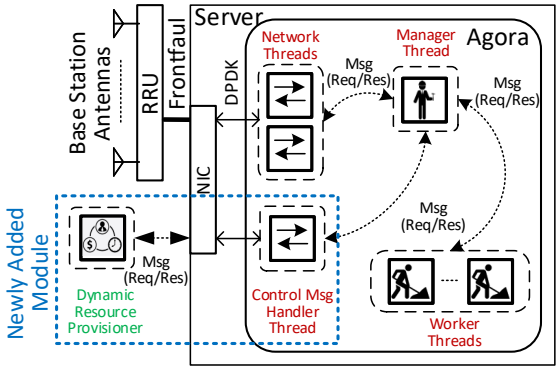

Figure 3: System Overview. DRP is the newly integrated module into Agora that allocates CPU cores on the fly at TTI granularity based on computational load.

Deployment. Q-Table is the primary input for the deployment. Similarly to training, the RL agent starts with random CPU cores and retrieves latency and spatial layer details from Agora to determine its state s. For this state s, the RL agent takes the best action from the Q-Table and tries to reach the optimal latency range.

3.3 Integrating RL-based DRP into Agora

We integrate our RL-based DRP into Agora as a new module that instructs the system to add, remove, or retain a CPU core based on its identified action. For a given cellular configuration, DRP targets allocating CPU cores such that the resulting TTI processing latency is within the optimal latency range marked in Figure 1.

Modifying Agora for Dynamic Load. To dynamically vary the computational load in Agora, we add a new feature to adapt the number of spatial layers across the TTIs to make it behave similarly to a real-world vRAN. In the current implementation, the number of spatial layers and the TTI in which these spatial layers change are arbitrarily chosen. In Agora, a profiling framework is added to capture the absolute latency in every TTI to facilitate DRP.

Implementation Details. The baseline Agora [3] is implemented in C++. We implement DRP as a separate module in Rust and integrate it into Agora. Agora fundamentally has three threads, namely, manager, worker, and network IO threads. A new control msg handler thread is now added in Agora and it serves all messages from DRP. The overview of Agora system with its updated design is illustrated in Figure 3. The updated Agora system with DRP is realized on an Intel(R) Xeon(R) Silver 4314 server operating at a maximum CPU clock of 2.40GHz. This server has 32 cores with support for AVX512 ISA, which is primarily required for efficient computation of baseband receiver functionalities.

3.4 Comparison Scheme

We design an SGD-based DRP algorithm as a simpler comparison scheme. SGD uses a gradient descent algorithm based on a Look-Up Table (LUT) to determine the optimal number of CPU cores. Each entry in the LUT has a direct mapping to the number of

Table 1: Agora Configuration

Parameter	Value
TTI & SNR	1 ms & 30 dB
Min and Max number of CPU Cores	3 and 15
Number of Base Station (BS) Antennas	64
Max number of spatial layers	16
Modulation & Code Rate	64 QAM & 0.85
Number of latency levels	11 (100 ms per level)

spatial layers. Similarly to RL, there is an obvious need for an offline exercise to populate this LUT for all possible spatial layers.

Mapping DRP as an SGD Problem. To map DRP as an SGD problem, the absolute processing latency and its corresponding spatial layer details from Agora must be integrated into the SGD algorithm. Let $t(c)$ represents the TTI processing latency as a function of the number of CPU cores c. Using $t(c)$ from Agora, we use the finite difference to estimate the gradient:

$$g_k = \frac{t(c_i + k + \delta) - t(c_i + k)}{\delta} \tag{2}$$

where c_i is the initial number of CPU cores, k is the CPU core change, and δ is the CPU core offset, which is typically set to 1.

Populating LUT and Deployment. The goal is to find c such that the resulting latency is within the optimal range. Initially, the SGD algorithm allocates min. CPU cores (c_i) and determines the corresponding TTI processing latency from Agora for specific spatial layers. For increasing k, it checks if the processing latency reaches within the optimal latency range. The same process is repeated for all possible spatial layers to fully populate LUT.

4 PRELIMINARY EVALUATION

We evaluate the performance of both RL- and SGD-based DRP algorithms. Our goal is to validate the feasibility of RL-based DRP for a massive MIMO vRAN.

Evaluation Setup. The cellular parameters used for this evaluation are listed in Table 1. The RL-based DRP is trained for 10,000 Episodes with a terminal count of 1000 Epochs.

Static Load Evaluations. To start with, we evaluate both versions of DRP for static configuration in Agora to see if the algorithms can find the CPU cores that satisfy the defined optimal latency range. For two different spatial layers (1 and 4), the latency versus CPU cores and the optimal CPU cores identified are plotted in Figure 4. Due to inter-CPU communication overhead in the system, baseband processing latency results in a *convex* function with respect to CPU cores, rather than decreasing monotonically. It is observed that both the RL- and SGD-based DRP algorithms successfully identify the optimal number of CPU cores, meeting the latency range target in Figure 1, in two different spatial layer scenarios.

Dynamic Load Evaluations. Next, we test them for varying computational load. Figure 5 shows real-time dynamic provisioning of CPU cores by RL- and SGD-based DRP algorithms. In this case, the DRP algorithms monitor the varying spatial layers across TTIs and continue to adapt to the optimal CPU cores. Initially, the number of spatial layers was four and the RL design took 34 ms in each core transition attempt, with a total of 68 ms for two attempts, to reach the optimal CPU cores of five. On the other hand, SGD took only 6 ms to reach the same optimal CPU cores. This trend follows

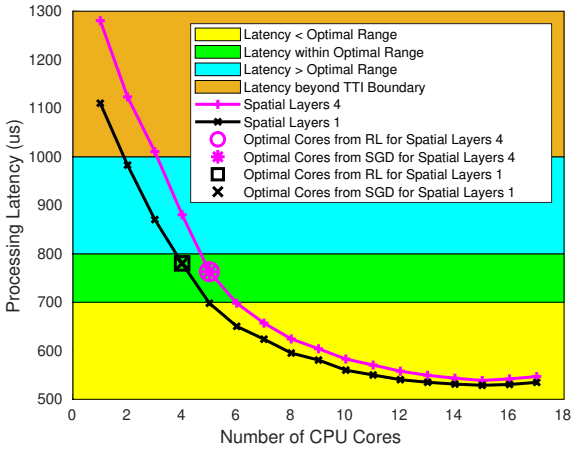

Figure 4: Processing latency across CPU cores. The optimal number of CPU cores is identified by SGD and RL algorithms for one and four MIMO spatial layers.

whenever there is a change in spatial layers across TTIs. The allocation time of 6 ms for SGD signifies that the cellular configuration in Agora should remain the same for at least 6 TTIs, whereas this number is very high and varies for RL.

We observe that SGD performs much faster than RL without loss of performance. To further analyze this, we characterize their key timing parameters. In the case of RL, the allocation time that defines the time required by DRP to assign optimal CPU cores is $(T_1 + T_2) \cdot E_1$, where T_1 is the number of TTIs required to complete the processing per CPU core attempt, T_2 is the TTIs per CPU core transition, and E_1 is the number of Epochs per Episode. Note that T_1 consists of $F_0 + F_1 + F_2 + F_3$, where F_0 is the number of TTIs to obtain Agora status, F_1 to estimate sample statistics [6], F_2 to compute mean latency [6], and F_3 for outliers. In our implementations, $F_0 = 3$, $F_1 = 10$, $F_2 = 15$ and $F_3 = 3$, while TTI per CPU core transition $T_2 = 3$. On the other hand, the allocation time of SGD is $F_0 + T_2$. Unlike RL, SGD does not need F_1, F_2, F_3 and Epoch iterations and, therefore, features a much shorter allocation time. We summarize the maximum allocation and training time in Table 2. Although the training time of RL is much longer than that of SGD, since it is an offline process, it may not have a major impact on deployment.

Memory Requirement. In the same table, we also compare their memory requirements. In the case of RL, the required memory is relatively high, since there are $15 \times 8 \times 11 = 1320$ possible states in Equation 1 and each state has Q-values for three different actions. However, the LUT in SGD contains only eight CPU cores information corresponding to eight spatial layers, resulting in a much smaller memory footprint.

5 DISCUSSION

The foregoing evaluations do *not* show compelling advantages of the RL-based DRP for a massive MIMO vRAN over the SGD one. However, this does not necessarily imply that the RL-based design fails. Indeed, its DRP is able to find the optimal CPU cores successfully, while simpler SGD can do so more efficiently. We argue that this is largely due to the inherent limitations of the current vRAN testbed (*i.e.,* limited dynamic nature). In other words, the

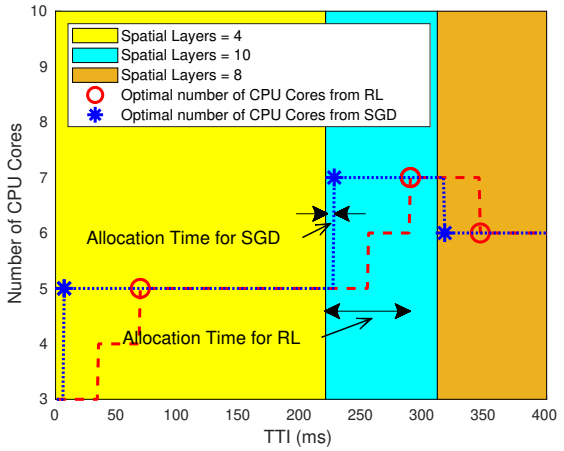

Figure 5: Real-time dynamic allocation of CPU cores by DRP algorithms for varying spatial layers across TTIs.

Table 2: Time and memory requirement comparison between RL- and SGD-based DRP algorithms.

Performance Metric	RL-based DRP	SGD-based DRP
Max. allocation time per fixed configuration	👎 (= 279 ms)	👍 (= 6 ms)
Max. training time	👍 (= 93 mins)	👍 (= 1.36 mins)
Memory requirement (elements)	👎 (= 1320×3)	👍 (= 8×2)

DRP problems were too simple under the current vRAN testbeds, and thus even simple SGD could perform well. In this regard, this section discusses limitations, future directions, and the challenges and opportunities of (RL-based) DRP for massive MIMO vRAN.

Limitation: Static Computation Load in Agora. Our exploration is limited to the current version of Agora, which works primarily for a fixed set of cellular parameters. In other words, Agora (and other vRAN testbeds) features only static or limited dynamic computation load, which imposes constraints on vRAN DRP research. For example, modulations, configured maximum Low-Density Parity-Check (LDPC) decoding iterations, SNRs, code rates, user mobilities, throughput requirements, operating bandwidths, and many more factors remain static across TTIs in Agora, though such factors frequently change in a real-world vRAN, directly affecting the computational load and the TTI processing time. Hardware-based testbeds feature a more dynamic load [1], but it is challenging to extend them to massive MIMO systems, and they are inflexible and difficult to program.

Need for Fully Dynamic MIMO vRAN Testbeds. In this work, as a first step in making software-based Agora more dynamic, we enabled the system to adapt spatial layers across TTIs. However, to evaluate DRP's full potential, we need a testbed that functions and behaves more reasonably like a real-world vRAN; Figure 6 experimentally demonstrates this. Figure 6 (*left*) illustrates that the processing time required for LDPC decoding of a code block for a spatial layer decreases as the SNR improves. For this reason, the overall TTI processing time shows an inversely proportional

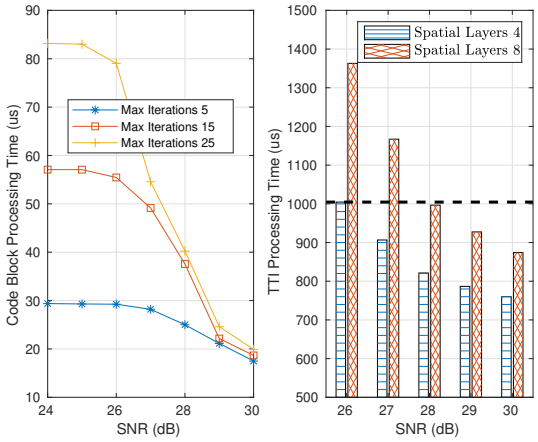

Figure 6: (*Left*): Processing time required to complete LDPC decoding of a code block for different values of the maximum iteration. (*Right*): TTI processing time across SNRs for four and eight spatial layers. Both figures demonstrate that factors other than spatial layers (*e.g.*, SNRs) can also critically affect the computational load and processing time, showing that fewer spatial layers could take longer TTI processing time.

relation to the SNR, as shown in Figure 6 (*right*), while it has a proportional relation to the spatial layers in Figure 4. The results imply that the applied model in this work that adapts only spatial layers is far from the ones required for massive MIMO vRAN DRP research, given that other cellular factors than spatial layers can also critically affect the vRAN computational load and processing time. Therefore, we need to extend the adapting feature to vary most of the cellular parameters in real time. However, it is challenging to model the compute behavior of a fully dynamic real-time vRAN system for the following reasons. First, there are no theoretical models that apply to how related factors vary in practical deployments. Second, these factors are often interdependent and their relationships could vary depending on communication scenarios [4]. In our future work, we will explore these challenges, aiming to enable fully dynamic Agora, as a *key enabler* of massive MIMO vRAN DRP research.

Promising Aspects of RL. We observe certain promising aspects of RL during our exploration. Specifically, RL is not getting trained for all theoretical states, as defined in Equation 1, but only for the legitimate states practically possible in real deployment. However, there is no such intelligence in SGD to find only the relevant and valid parameter combination possible in a real-world vRAN; SGD populates the LUT for all possible combinations of parameters, irrespective of whether a particular combination is realistic. Thus, RL is a more automated and self-learning process, while SGD requires human intervention in populating the LUT. Moreover, when many factors become dynamic, the CPU core-latency convex relationship in Figure 4 is likely to become non-convex, where simple SGD could fail and automated learning methods such as RL might become critical. Furthermore, the RL approach provides flexibility in terms of reward designs (beyond the optimal latency range, *e.g.*, for ultra-reliable communications), and the allocation time of the RL method can be significantly reduced by optimizing communications between Agora and the RL DRP module; this communication

overhead is a possible root cause of the high latency observed in RL. However, quantifying these potentials of RL requires additional experimental studies and validations.

Advanced RL Design for DRP. Our initial RL design for DRP is a simplistic version because of the limited adaptability of the dynamic parameters in Agora. When fully dynamic vRAN testbeds become available, we will expand the state space in Equation 1 to include more parameters of the vRAN. For example, when the Modulation and Coding Scheme (MCS) becomes adaptable, a new term that involves m (*e.g.,* $m \cdot L \cdot U \cdot C$) should be added to Equation 1, where m represents the MCS used in the current TTI. Similarly, as more parameters become adaptable, the state space will expand to facilitate RL to learn the computational load behavior of vRAN for every possible combination of dynamic parameters. As more parameters become adaptable, the state space will grow exponentially and soon result in *curse of dimentionality* problem, increasing the memory footprint. To solve this problem, we plan to deploy techniques such as meta-learning and hierarchical RL, to reduce the effective state space and memory. Furthermore, allocating a specific number of CPU cores directly, rather than adding/removing a single core at a time, can be the action of the RL agent in the improved design to reduce the allocation time to quickly transition to optimal cores. In our future work, we will also consider a design that jointly optimizes the cores (latency) and the spatial layers (throughput).

DRP for Parallel Non-Linear MIMO Processing. To further increase MIMO spatial layers and thus the network performance, researchers started to explore parallel architectures for MIMO processing. This approach aims to leverage near-optimal but computationally demanding *non-linear* algorithms. The key idea is to transform their heavy task into multiple easier subtasks that can be processed in parallel. Compared to conventional linear MIMO processing, these *non-linear* algorithms can greatly enhance network throughput within a reasonable latency, leveraging the trade-off between achievable MIMO performance and resources [8]. In addition to the vRAN latency performance (which this work focuses on), such a trade-off needs to be considered simultaneously, thus making DRP in vRAN more challenging but also more interesting.

6 RELATED WORK

Massive MIMO vRAN Solutions. Due to its potential, vRAN is an active area of research, and many interesting problems are being looked into in both academia and industry to improve overall network performance. Hydra [5] addressed the critical challenges of scaling massive MIMO systems. The authors proposed a distributed architecture for baseband processing that partitions the computational load across multiple processing units. However, as the number of antennas increases, due to their manual partitioning scheme, resource allocation shall be challenging. Slingshot [9] framework improves the reliability and fault tolerance of baseband processing in vRAN. Ensure continuous service in case of hardware failures, network disruptions, or software bugs, which are more prevalent in vRANs.

Emerging vRAN Works. An approach was studies to manage the computational demands of base stations by leveraging data center resources [2]. However, it did not consider important factors such as load and SNR variations. The authors proposed a framework that offloads baseband processing tasks from individual base stations to a centralized data center for scalability and flexibility. Violinn [12] introduced the concept of a spare edge device to handle sudden load variations, without having to continuously over-provision. Several spare device selection algorithms with different cost and performance trade-offs were also proposed. A deep learning technique was studied to optimize the computing and radio resources in vRANs [1]. Although these approaches enabled a more efficient management of resources, none of them supports MIMO.

7 CONCLUSION

We conclude that more experimental studies are required to demonstrate the need for the definitive use of RL in DRP for massive MIMO vRAN. We discuss a path forward in realizing a fully dynamic Agora and the adaptations required in DRP research. In general, the resource provision task tends to be a more complicated problem in vRAN as wireless networks evolve. Given this trend, we anticipate that more comprehensive approaches like RL will eventually be required for DRP.

ACKNOWLEDGEMENTS

We thank our shepherd Renjie Zhao, the anonymous reviewers of this paper, and members of the Yale Efficient Computing Lab (YECL) for their insightful comments. This research is supported in part by the NSF Awards #2211945 and #2147946.

REFERENCES

[1] J. A. Ayala-Romero, A. Garcia-Saavedra, M. Gramaglia, X. Costa-Perez, A. Banchs, J. J. Alcaraz. vrain: A deep learning approach tailoring computing and radio resources in virtualized rans. *MobiCom '19: The 25th Annual International Conference on Mobile Computing and Networking*. Association for Computing Machinery, New York, NY, USA, 2019. ISBN 9781450361699.

[2] S. Bhaumik, S. P. Chandrabose, M. K. Jataprolu, G. Kumar, A. Muralidhar, P. Polakos, V. Srinivasan, T. Woo. Cloudiq: a framework for processing base stations in a data center. *Proceedings of the 18th Annual International Conference on Mobile Computing and Networking*, Mobicom '12, 125–136. Association for Computing Machinery, New York, NY, USA, 2012. ISBN 9781450311595.

[3] J. Ding, R. Doost-Mohammady, A. Kalia, L. Zhong. Agora: Real-time massive mimo baseband processing in software. *Proceedings of the 16th International Conference on emerging Networking EXperiments and Technologies*, 232–244, 2020.

[4] X. Foukas, B. Radunovic. Concordia: teaching the 5g vran to share compute. *Proceedings of the 2021 ACM SIGCOMM 2021 Conference*, SIGCOMM '21, 580–596. Association for Computing Machinery, New York, NY, USA, 2021. ISBN 9781450383837. doi:10.1145/3452296.3472894.

[5] J. Gong, A. Kalia, M. Yu. Scalable distributed massive MIMO baseband processing. *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*, 405–417. USENIX Association, Boston, MA, 2023. ISBN 978-1-939133-33-5.

[6] R. Jain. *The Art of Computer Systems Performance Analysis: Techniques for Experimental Design, Measurement, Simulation, and Modeling.* John Wiley Sons, Inc, 1991. ISBN 978-0-471-50336-1.

[7] M. Khani, S. Jamali, M. K. Sohrabi, M. M. Sadr, A. Ghaffari. Resource allocation in 5g cloud-ran using deep reinforcement learning algorithms: A review. *Trans. Emerg. Telecommun. Technol.*, **35**(1), 2024. ISSN 2161-3915.

[8] M. Kim, S. Mandrà, D. Venturelli, K. Jamieson. Physics-inspired heuristics for soft mimo detection in 5g new radio and beyond. *Proceedings of the 27th Annual International Conference on Mobile Computing and Networking*, 42–55, 2021.

[9] N. Lazarev, T. Ji, A. Kalia, D. Kim, I. Marinos, F. Y. Yan, C. Delimitrou, Z. Zhang, A. Akella. Resilient baseband processing in virtualized rans with slingshot. *Proceedings of the ACM SIGCOMM 2023 Conference*, ACM SIGCOMM '23, 654–667. Association for Computing Machinery, New York, NY, USA, 2023. ISBN 9798400702365.

[10] P. Rost, A. Maeder, M. C. Valenti, S. Talarico. Computationally aware sum-rate optimal scheduling for centralized radio access networks. *2015 IEEE Global Communications Conference (GLOBECOM)*, 1–6, 2015.

[11] R. S. Sutton, A. G. Barto. *Reinforcement Learning: An Introduction.* The MIT Press, second edn., 2018.

[12] K. Toczé, A. J. Fahs, G. Pierre, S. Nadjm-Tehrani. Violinn: Proximity-aware edge placementwith dynamic and elastic resource provisioning. *ACM Trans. Internet Things*, **4**(1), 2023.